



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

Computer Architecture

Chapter 3

Basic Computer Design – part 2

Afarghadan@aut.ac.ir

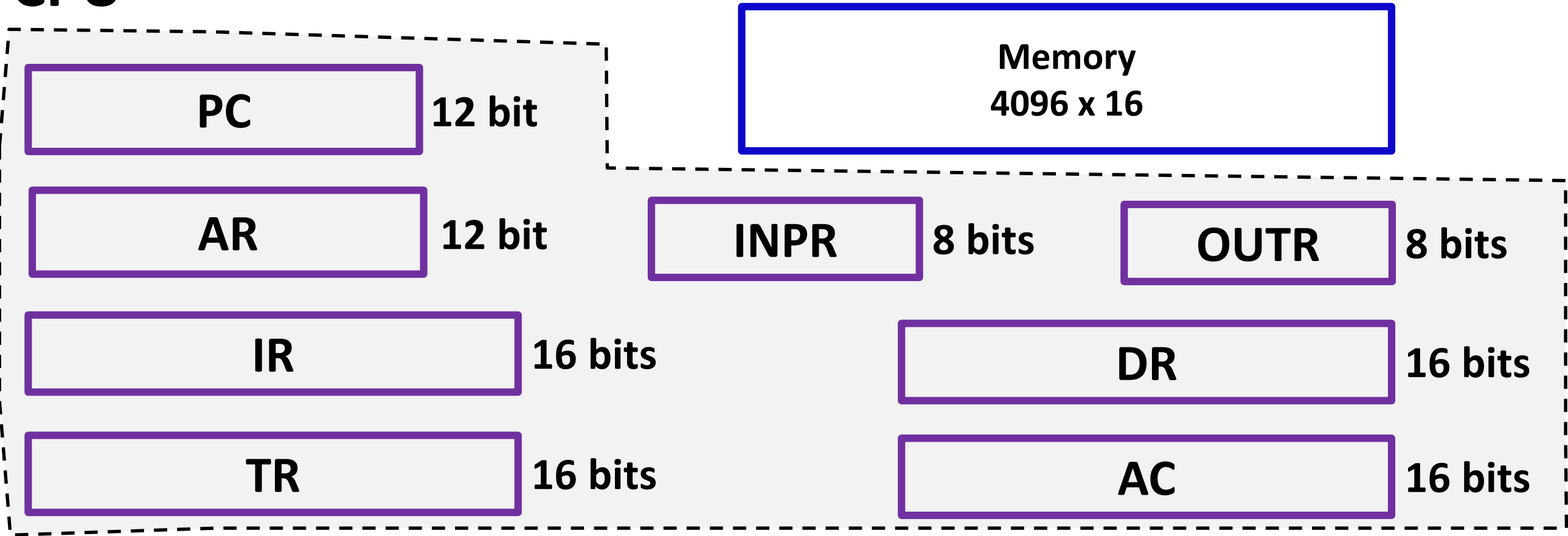
عظیم فرقدان



- 1 Recap**
- 2 Basic Computer Instructions**
- 3 Timing and Control**
- 4 Instruction Cycle**
- 5 Memory Reference Instructions**
- 6 Input-Output and Interrupt**
- 7 Complete Computer Description**
- 8 Design of Basic Computer**
- 9 Design of Accumulator Logic**

Recap: Basic Computer Registers

CPU

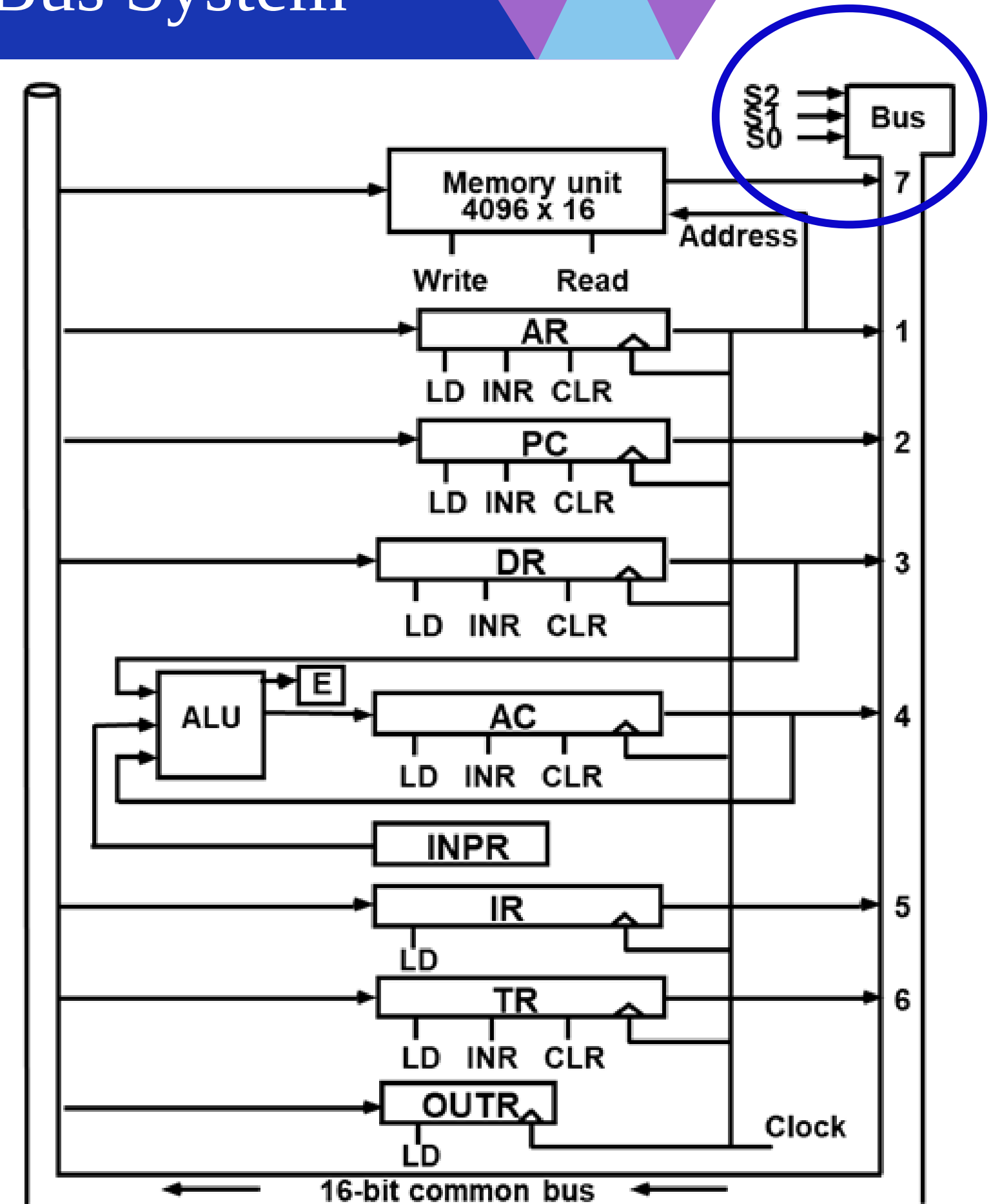


Holds memory operand	Data Register	16	DR
Holds address for memory	Address Register	12	AR
Process register	Accumulator	16	AC
Holds instruction code	Instruction Register	16	IR
Holds address of instruction	Program Counter	12	PC
Holds temporary data	Temporary Register	16	TR
Holds input character	Input Register	8	INPR
Holds output character	Output Register	8	OUT R

Recap: Common Bus System

- The registers in the basic computer are connected using a **bus**. This helps **simplify the circuitry** over complete connections between registers.

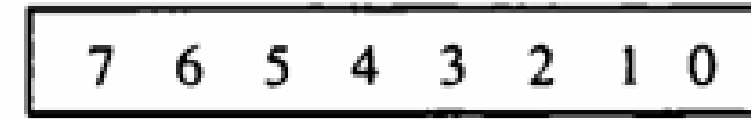
S0	S1	S2	Register
0	0	0	X
0	0	1	AR
0	1	0	PC
0	1	1	DR
1	0	0	AC
1	0	1	IR
1	1	0	TR
1	1	1	Memory



Recap: Register Transfer Language



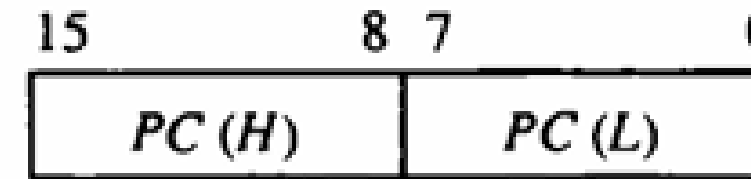
(a) Register R



(b) Showing individual bits



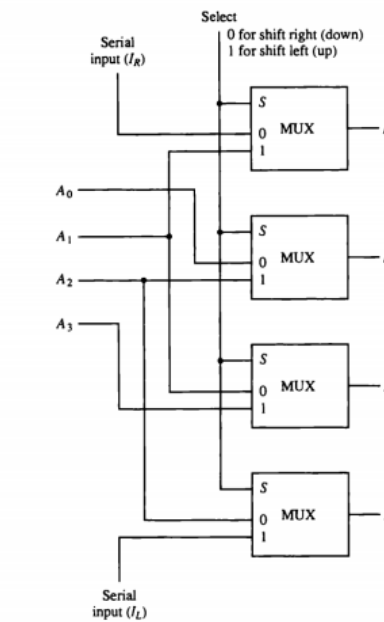
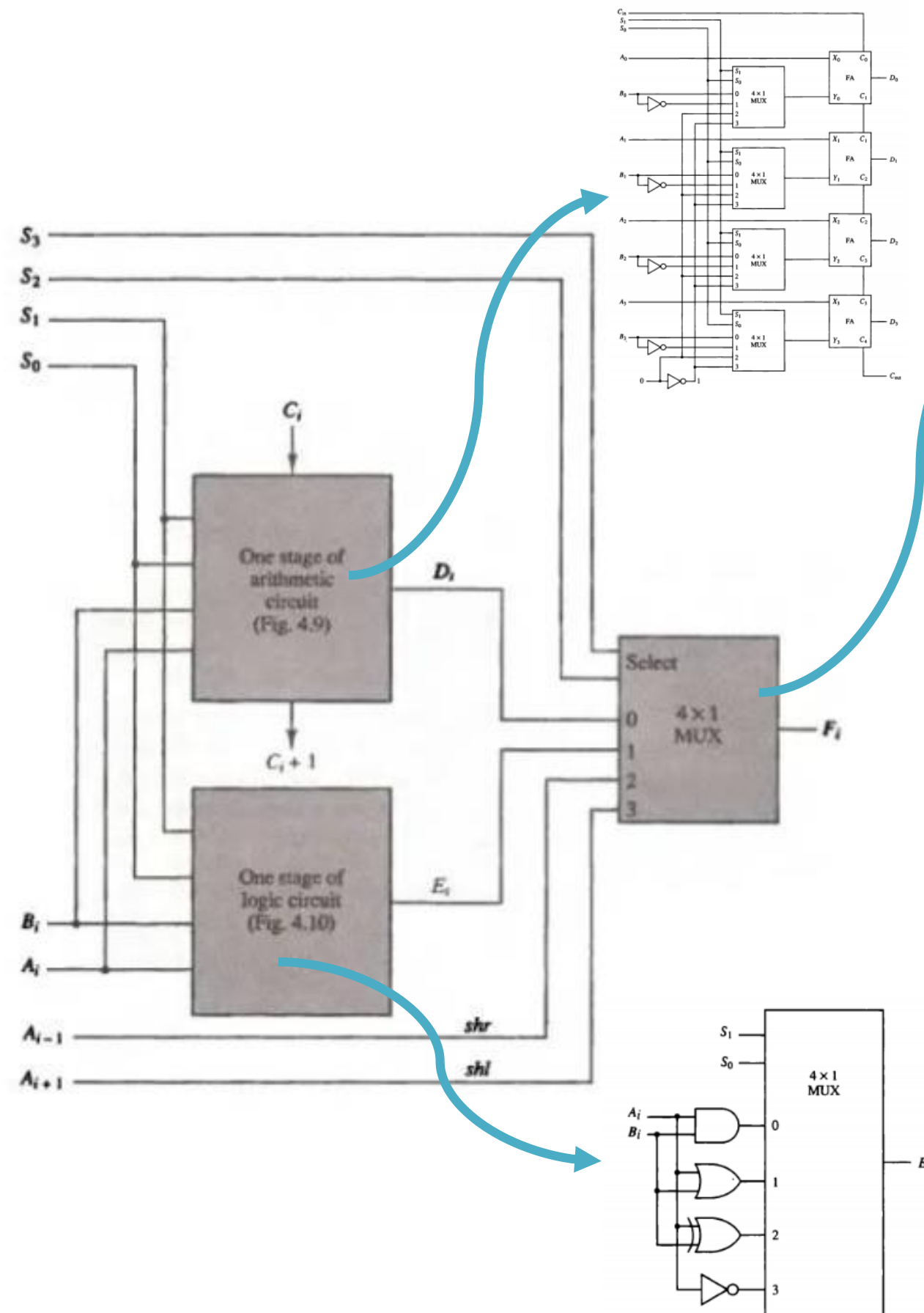
(c) Numbering of bits



(d) Divided into two parts

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	$MAR, R2$
Parentheses ()	Denotes a part of a register	$R2(0-7), R2(L)$
Arrow $\leftarrow$	Denotes transfer of information	$R2 \leftarrow R1$
Comma ,	Separates two microoperations	$R2 \leftarrow R1, R1 \leftarrow R2$

Recap: Arithmetic Logic Unit



Operation select					Operation	Function
S_3	S_2	S_1	S_0	C_{in}		
0	0	0	0	0	$F = A$	Transfer A
0	0	0	0	1	$F = A + 1$	Increment A
0	0	0	1	0	$F = A + B$	Addition
0	0	0	1	1	$F = A + B + 1$	Add with carry
0	0	1	0	0	$F = A + \overline{B}$	Subtract with borrow
0	0	1	0	1	$F = A + \overline{B} + 1$	Subtraction
0	0	1	1	0	$F = A - 1$	Decrement A
0	0	1	1	1	$F = A$	Transfer A
0	1	0	0	x	$F = A \wedge B$	AND
0	1	0	1	x	$F = A \vee B$	OR
0	1	1	0	x	$F = A \oplus B$	XOR
0	1	1	1	x	$F = \overline{A}$	Complement A
1	0	x	x	x	$F = shr A$	Shift right A into F
1	1	x	x	x	$F = shl A$	Shift left A into F

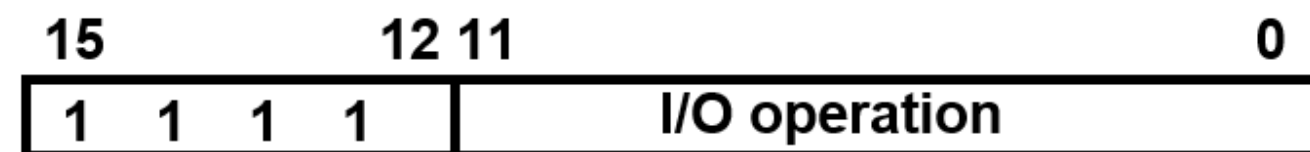
Memory-Reference Instructions (OP-code = 000 ~ 110)



Register-Reference Instructions (OP-code = 111, I = 0)



Input-Output Instructions (OP-code = 111, I = 1)



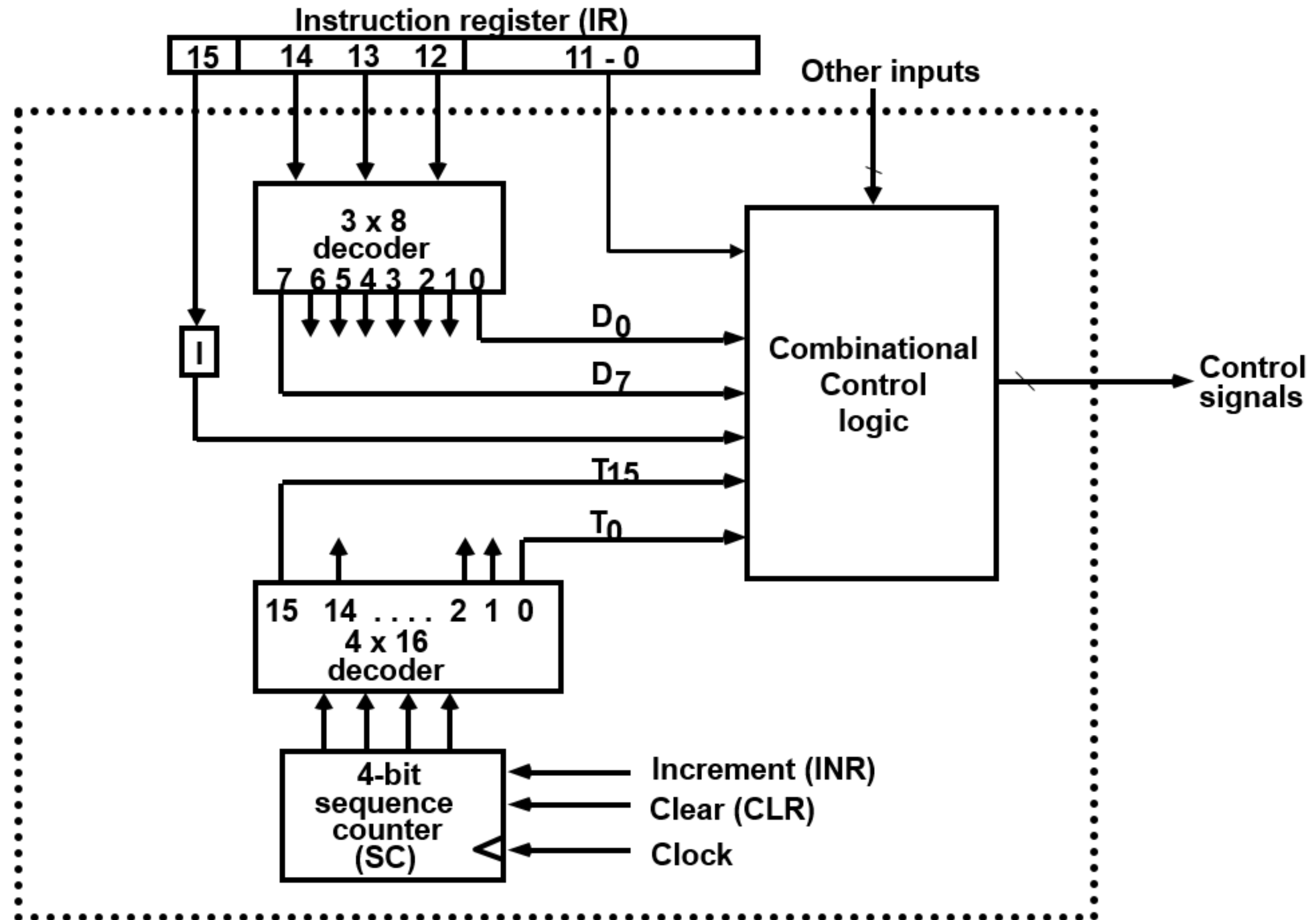
Basic Computer Instructions

Symbol	Hex Code		Description
	I = 0	I = 1	
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load AC from memory
STA	3xxx	Bxxx	Store content of AC into memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instr. if AC is positive
SNA	7008		Skip next instr. if AC is negative
SZA	7004		Skip next instr. if AC is zero
SZE	7002		Skip next instr. if E is zero
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

- A computer should have a set of instructions so that the user can construct **machine language programs** to **evaluate** any **function** that is known to be **computable**.
- **Instruction types:**
 - Functional Instructions
 - Arithmetic, logic, and shift instructions → ADD, CMA, INC, CIR, CIL, AND, CLA
 - Transfer Instructions
 - Data transfers between the main memory and processor registers → LDA, STA
 - Control Instructions
 - Program sequencing and control → BUN, BSA, ISZ
 - Input/output Instructions
 - Input and output → INP, OUT

- **Control unit (CU)** of a processor translates from machine instructions to the control signals for the **microoperations** that implement them.
- Control units are implemented in one of two ways
 - **Hardwired** Control
 - CU is made up of **sequential and combinational circuits** to generate the control signals
 - **Microprogrammed** Control
 - A control memory on the processor contains **microprograms** that activate the necessary control signals
- We will consider a **hardwired** implementation for the Basic Computer.

Control Unit

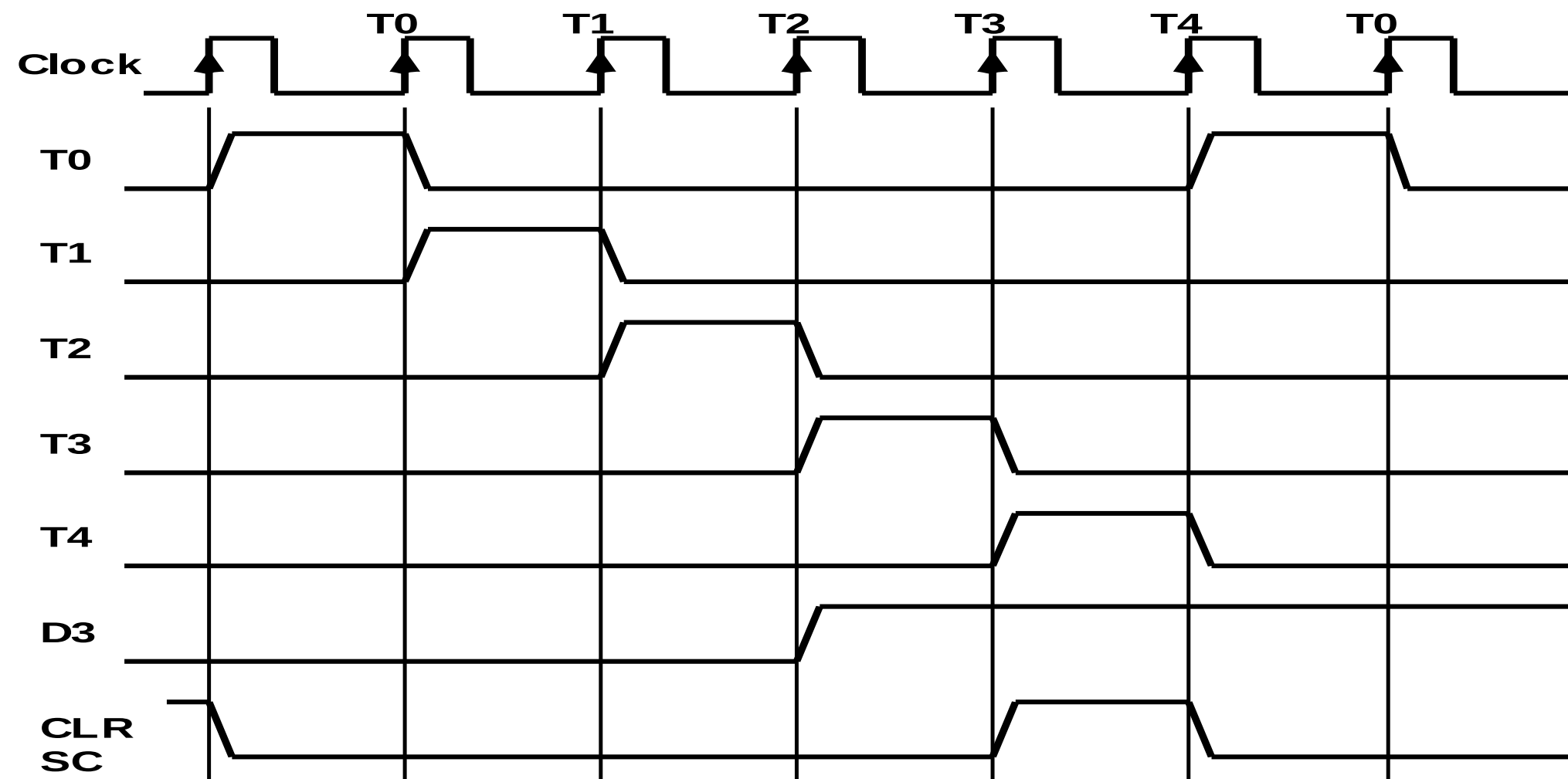


- The timing for all registers in the basic computer is controlled by a **master clock** generator.
- The **clock pulses** are applied to all flip-flops and registers in the control unit.
- The clock pulses do not change the state of a register unless the register is **enabled** by a **control signal**.
- The **control signals** are generated in the **control unit** and provide **control inputs** for the multiplexers in the **common bus**, control inputs in **processor registers** and microoperations for **accumulator**.

Timing Signals

- Generated by 4-bit **sequence counter** and 4x16 decoder.
- The SC can be incremented or cleared.
- Example: T0, T1, T2, T3, T4, T0, T1, ...
- Assume: At time T4, SC is cleared to 0 if decoder output D3 is active.

$D_3T_4: SC \leftarrow 0$

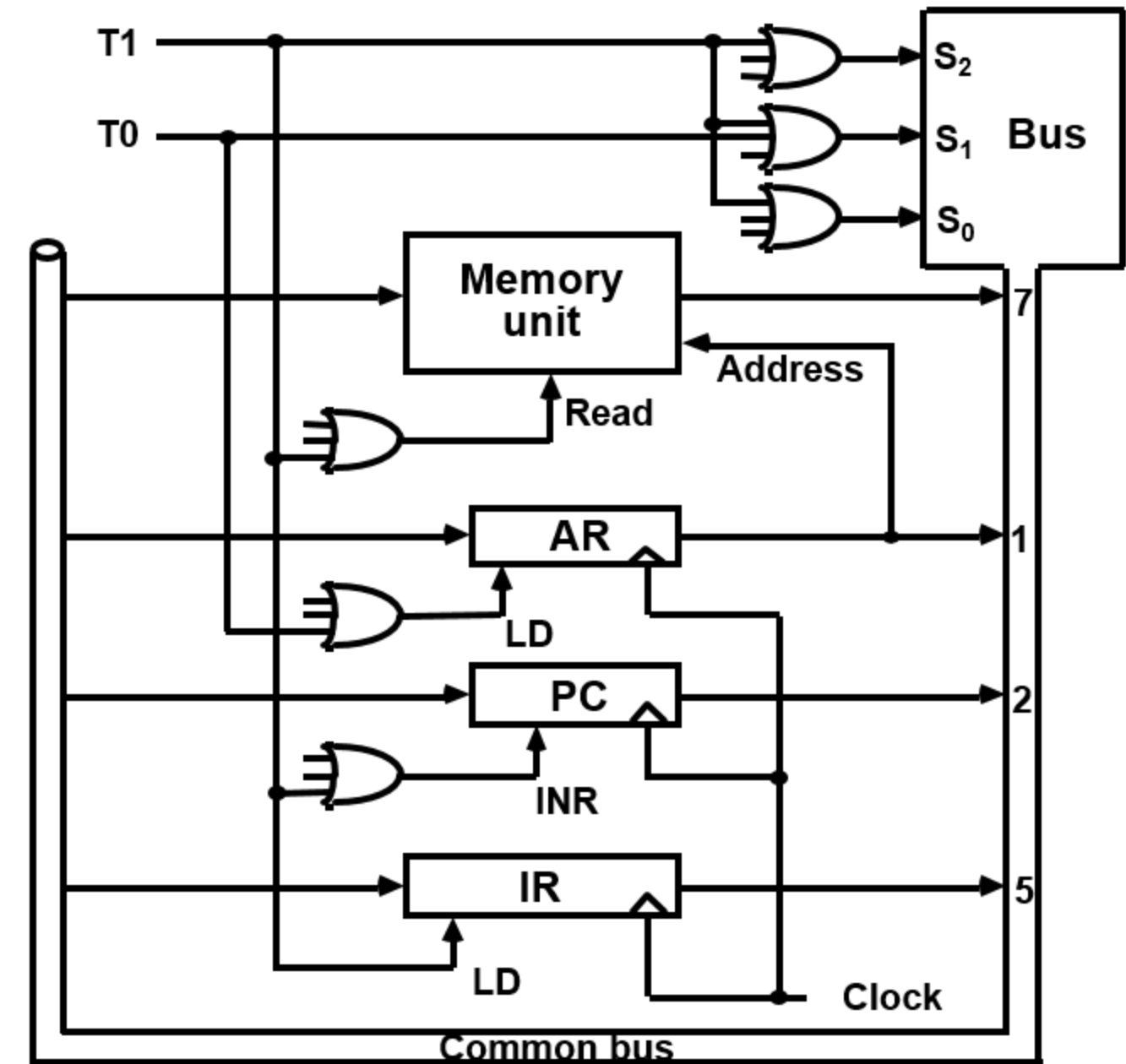


- In basic computer, a machine instruction is executed in the following cycle:
 - 1 **Fetch** an instruction from memory
 - 2 **Decode** the instruction
 - 3 **Read** the effective **address** from memory if the instruction has an **indirect** address
 - 4 **Execute** the instruction
- After an instruction is executed, the cycle starts again at step 1 for the **next instruction** unless a HALT instruction is encountered.
- Every **different** processor has its own (different) **instruction cycle**.

Fetch and Decode

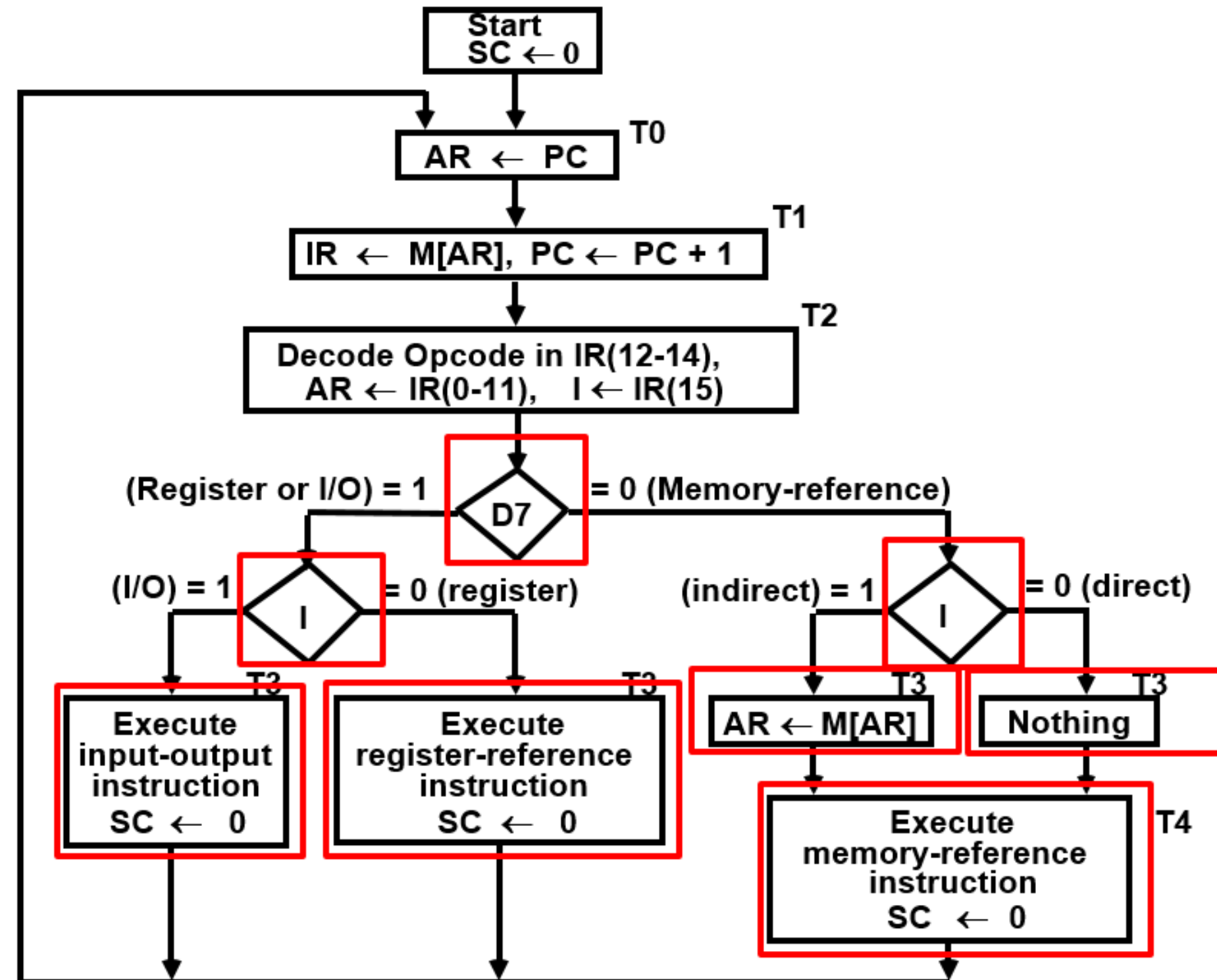
- T0: $AR \leftarrow PC$ ($S_0S_1S_2 = 010$, $T_0 = 1$)
- T1: $IR \leftarrow M[AR]$, $PC \leftarrow PC + 1$ ($S_0S_1S_2 = 111$, $T_1 = 1$)
- T2: $D_0, \dots, D_7 \leftarrow \text{Decode } IR(12-14)$, $AR \leftarrow IR(0-11)$, $I \leftarrow IR(15)$

- Initially, the PC is loaded with the address of the first instruction in the program. The SC is cleared to 0. After each clock pulse, SC is incremented by one, so that the timing signals go through a sequence T_0 , T_1 , T_2 , and so on.



Determine the Type of Instruction

- During T_3 the control unit determines the type of instruction that was just read from memory.



Memory Reference Instructions

Symbol	Operation Decoder	Symbolic Description
AND	D ₀	$AC \leftarrow AC \wedge M[AR]$
ADD	D ₁	$AC \leftarrow AC + M[AR], E \leftarrow C_{out}$
LDA	D ₂	$AC \leftarrow M[AR]$
STA	D ₃	$M[AR] \leftarrow AC$
BUN	D ₄	$PC \leftarrow AR$
BSA	D ₅	$M[AR] \leftarrow PC, PC \leftarrow AR + 1$
ISZ	D ₆	$M[AR] \leftarrow M[AR] + 1, \text{ if } M[AR] + 1 = 0 \text{ then } PC \leftarrow PC + 1$

- The effective address of the instruction is in AR and was placed there during timing signal T₂ when I=0, or during timing signal T₃ when I=1
- Memory cycle is assumed to be short enough to complete in a CPU cycle
- The execution of MR instruction starts with T₄
- AND to AC
 - D₀T₄: $DR \leftarrow M[AR]$ Read operand
 - D₀T₅: $AC \leftarrow AC \wedge DR, SC \leftarrow 0$ AND with AC

- ADD to AC

- $D_1T_4: DR \leftarrow M[AR]$

Read operand

- $D_1T_5: AC \leftarrow AC + DR, E \leftarrow C_{out}, SC \leftarrow 0$

Add to AC and store carry in E

- LDA: Load to AC

- $D_2T_4: DR \leftarrow M[AR]$

- $D_2T_5: AC \leftarrow DR, SC \leftarrow 0$

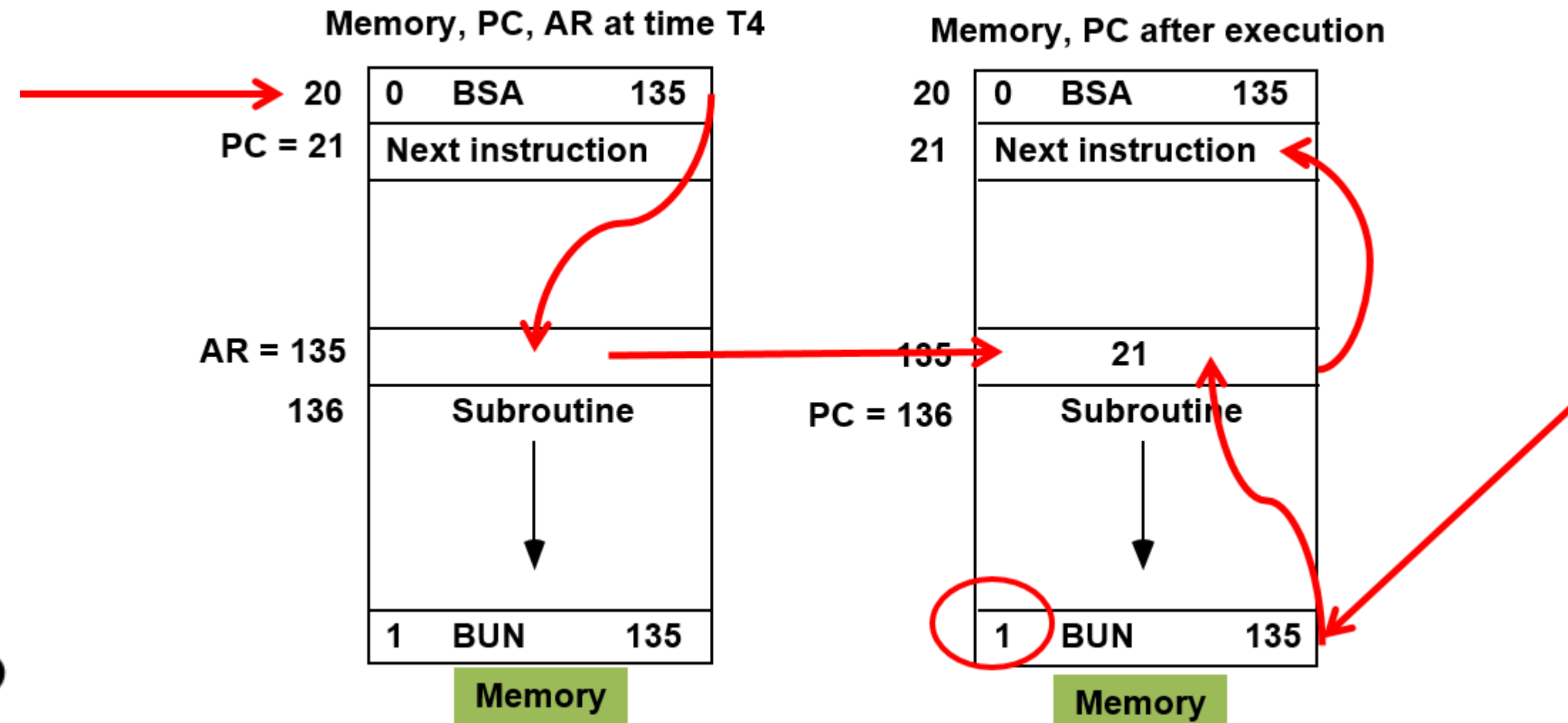
- BUN: Branch Unconditionally

- $D_4T_4: PC \leftarrow AR, SC \leftarrow 0$

Memory Reference Instructions

■ BSA

- $D_5T_4: M[AR] \leftarrow PC, AR \leftarrow AR + 1$
- $D_5T_5: PC \leftarrow AR, SC \leftarrow 0$

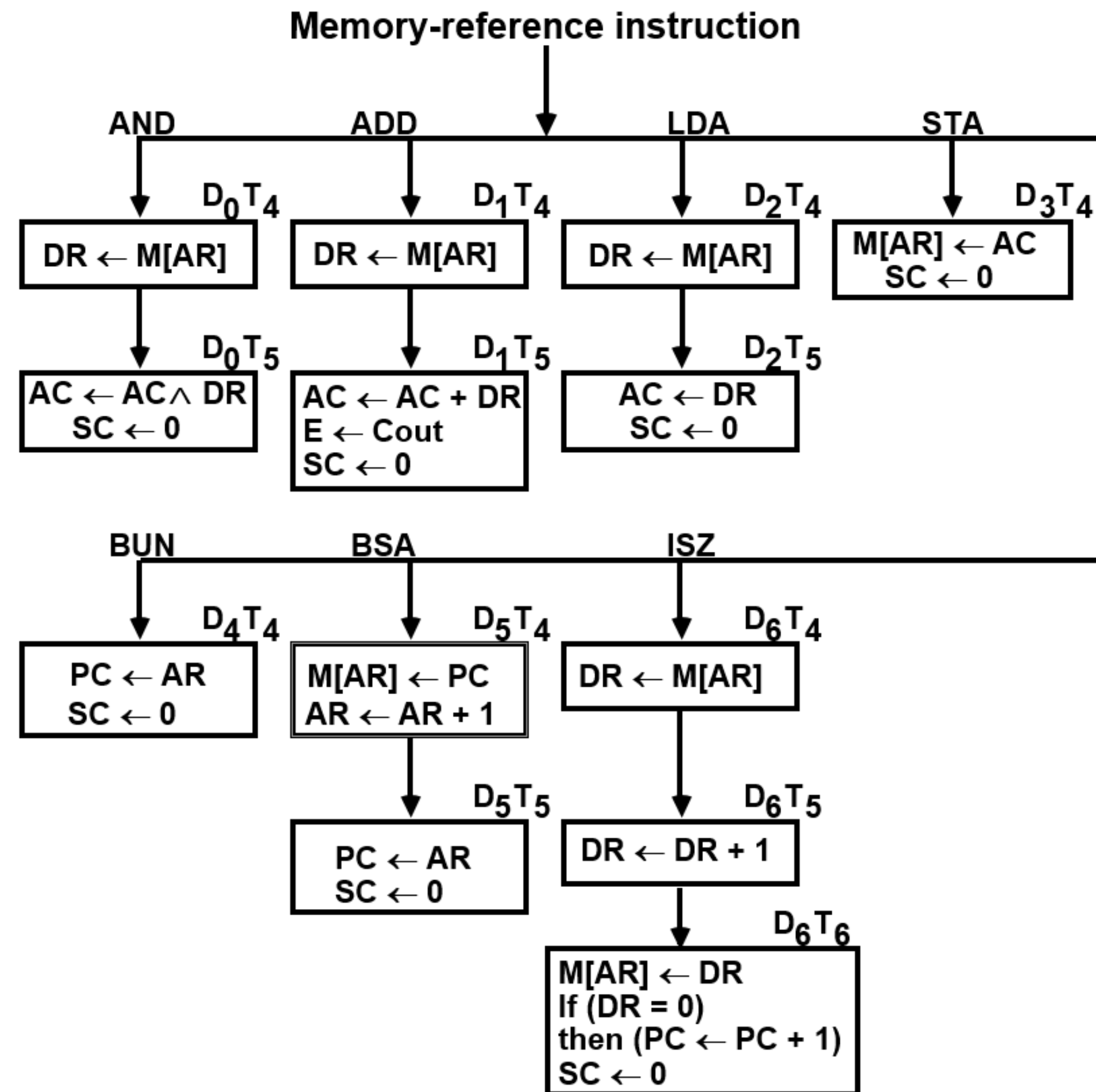


■ ISZ: Increment and Skip-if-Zero

- $D_6T_4: DR \leftarrow M[AR]$
- $D_6T_5: DR \leftarrow DR + 1$

$D_6T_6 :$ $M[AR] \leftarrow DR$
If $(DR = 0)$
then $(PC \leftarrow PC + 1)$
 $SC \leftarrow 0$

Memory Reference Instructions Flowchart

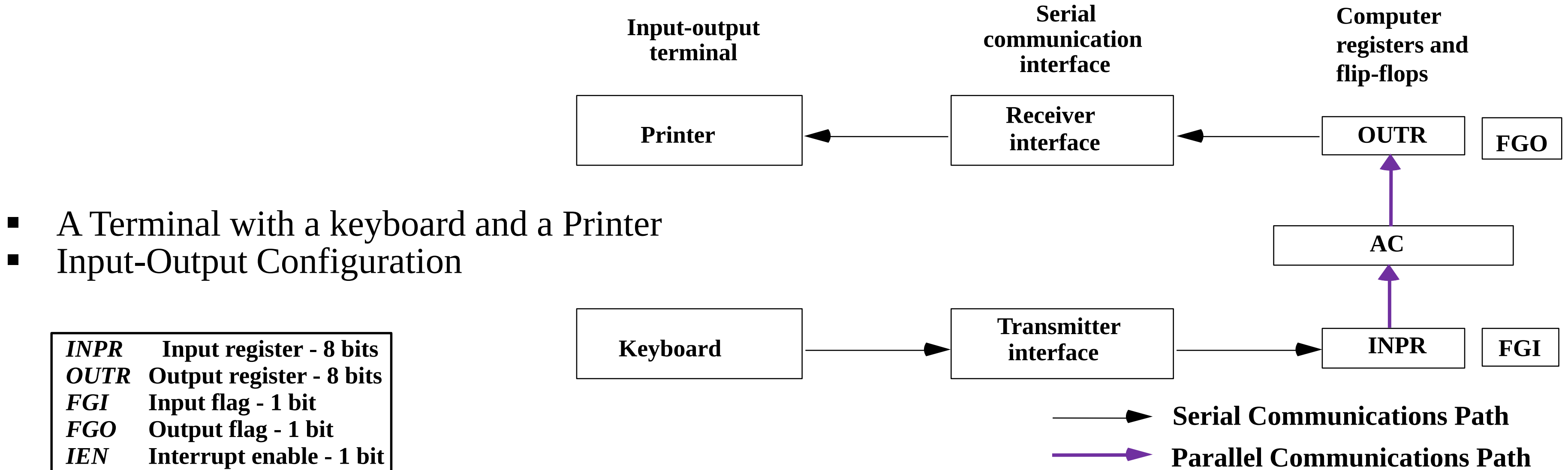


Register Reference Instructions

- Register Reference Instructions are identified when
 - $D_7 = 1, I = 0$
 - Register Ref. Instr. is specified in $b_0 \sim b_{11}$ of IR
 - Execution starts with timing signal T_3
- $R = D_7I'T_3 \Rightarrow$ Register Reference Instruction
- $B_i = IR(i), i=0,1,\dots,11$

	$r:$	$SC \leftarrow 0$	Clear SC
CLA	$rB_{11}:$	$AC \leftarrow 0$	Clear AC
CLE	$rB_{10}:$	$E \leftarrow 0$	Clear E
CMA	$rB_9:$	$AC \leftarrow \overline{AC}$	Complement AC
CME	$rB_8:$	$E \leftarrow \overline{E}$	Complement E
CIR	$rB_7:$	$AC \leftarrow \text{shr } AC, AC(15) \leftarrow E, E \leftarrow AC(0)$	Circulate right
CIL	$rB_6:$	$AC \leftarrow \text{shl } AC, AC(0) \leftarrow E, E \leftarrow AC(15)$	Circulate left
INC	$rB_5:$	$AC \leftarrow AC + 1$	Increment AC
SPA	$rB_4:$	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$	Skip if positive
SNA	$rB_3:$	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$	Skip if negative
SZA	$rB_2:$	If $(AC = 0)$ then $PC \leftarrow PC + 1$	Skip if AC zero
SZE	$rB_1:$	If $(E = 0)$ then $(PC \leftarrow PC + 1)$	Skip if E zero
HLT	$rB_0:$	$S \leftarrow 0$ (S is a start-stop flip-flop)	Halt computer

Input-Output and Interrupt



- The terminal sends and receives **serial** information
- The **serial** info. from the keyboard is shifted into INPR
- The **serial** info. for the printer is stored in the OUTR
- INPR and OUTR communicate with the terminal **serially** and with the AC in **parallel**.
- The flags are needed to **synchronize** the timing difference between I/O device and the computer

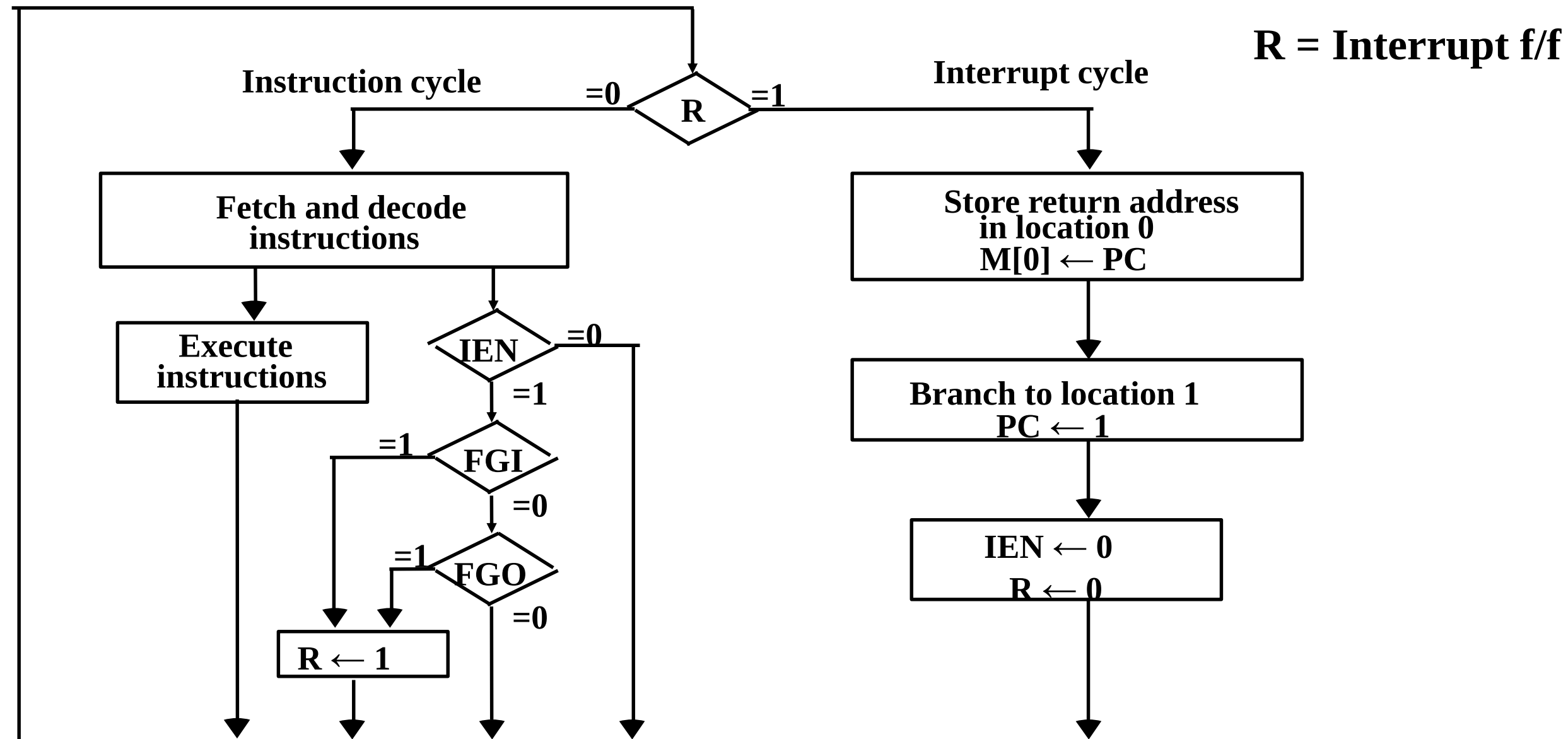
- $D_7IT_3 = p$
- $IR(i) = B_i, i = 6, \dots, 11$

INP	p:	$SC \leftarrow 0$	Clear SC
OUT	pB_{11} :	$AC(0-7) \leftarrow INPR, FGI \leftarrow 0$	Input char. to AC
SKI	pB_{10} :	$OUTR \leftarrow AC(0-7), FGO \leftarrow 0$	Output char. from AC
SKO	pB_9 :	if($FGI = 1$) then ($PC \leftarrow PC + 1$)	Skip on input flag
ION	pB_8 :	if($FGO = 1$) then ($PC \leftarrow PC + 1$)	Skip on output flag
IOF	pB_7 :	$IEN \leftarrow 1$	Interrupt enable on
	pB_6 :	$IEN \leftarrow 0$	Interrupt enable off

- Open communication only when some data has to be passed → interrupt.
- The I/O interface, instead of the CPU, monitors the I/O device.
- When the interface finds that the I/O device is ready for data transfer, it generates an interrupt request to the CPU
- Upon detecting an interrupt, the CPU stops momentarily the task it is doing, branches to the service routine to process the data transfer, and then returns to the task it was performing.
- IEN (Interrupt-enable flip-flop)
 - Can be set and cleared by instructions
 - When cleared, the computer cannot be interrupted

Interrupt Cycle Flowchart

- The interrupt cycle is a HW implementation of a branch and save return address operation.
- At the beginning of the **next instruction cycle**, the instruction that is read from memory is **in address 1**.
- At memory address 1, the programmer must store a branch instruction that sends the control to an **interrupt service routine (ISR)**
- The instruction that returns the control to the original program is "indirect BUN 0"



■ Register Transfer Statements for Interrupt Cycle

- $R \leftarrow 1$ if $IEN (FGI + FGO)T_0'T_1'T_2'$
 $\Leftrightarrow T_0'T_1'T_2' (IEN)(FGI + FGO): R \leftarrow 1$

■ The **fetch and decode** phases of the instruction cycle **must be modified**

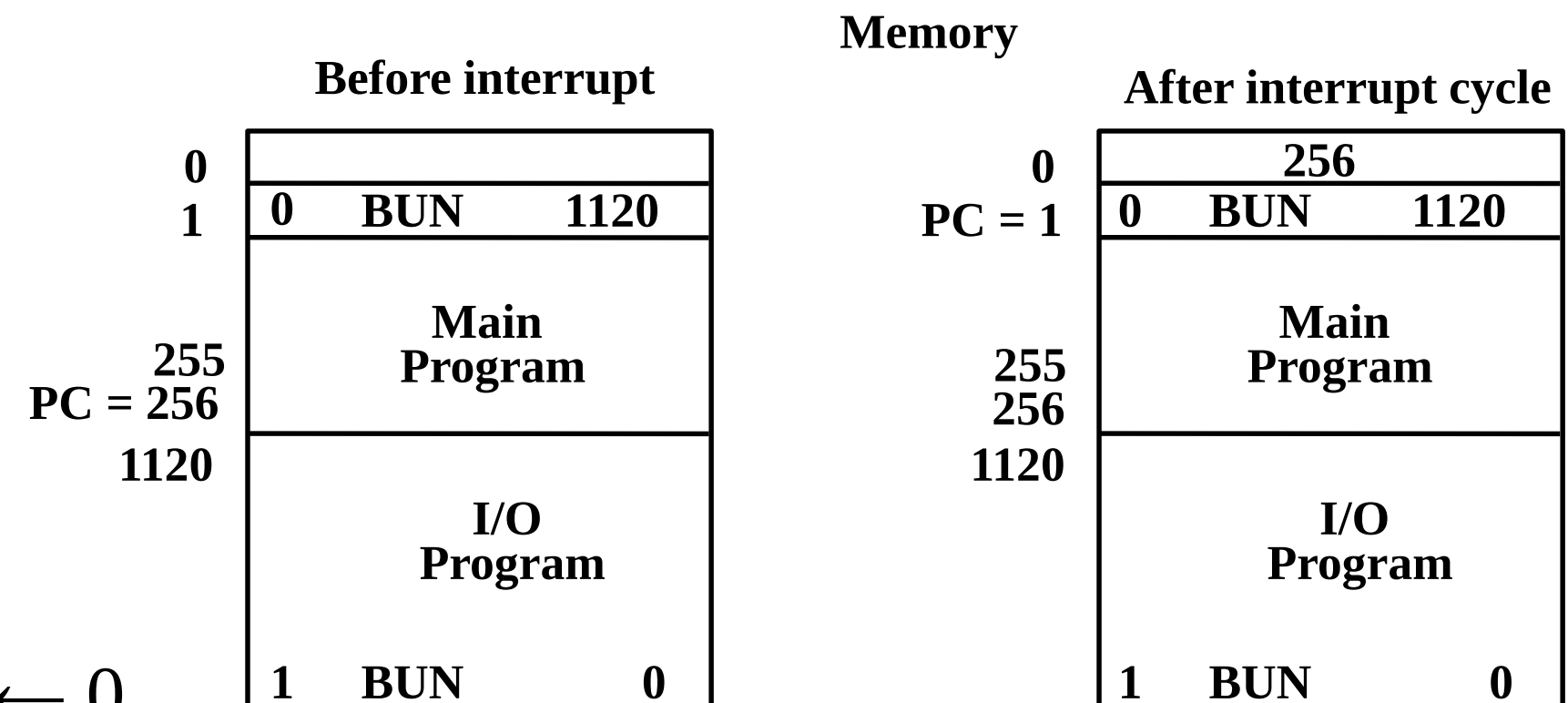
- Replace T_0, T_1, T_2 with $R'T_0, R'T_1, R'T_2$

■ The interrupt cycle:

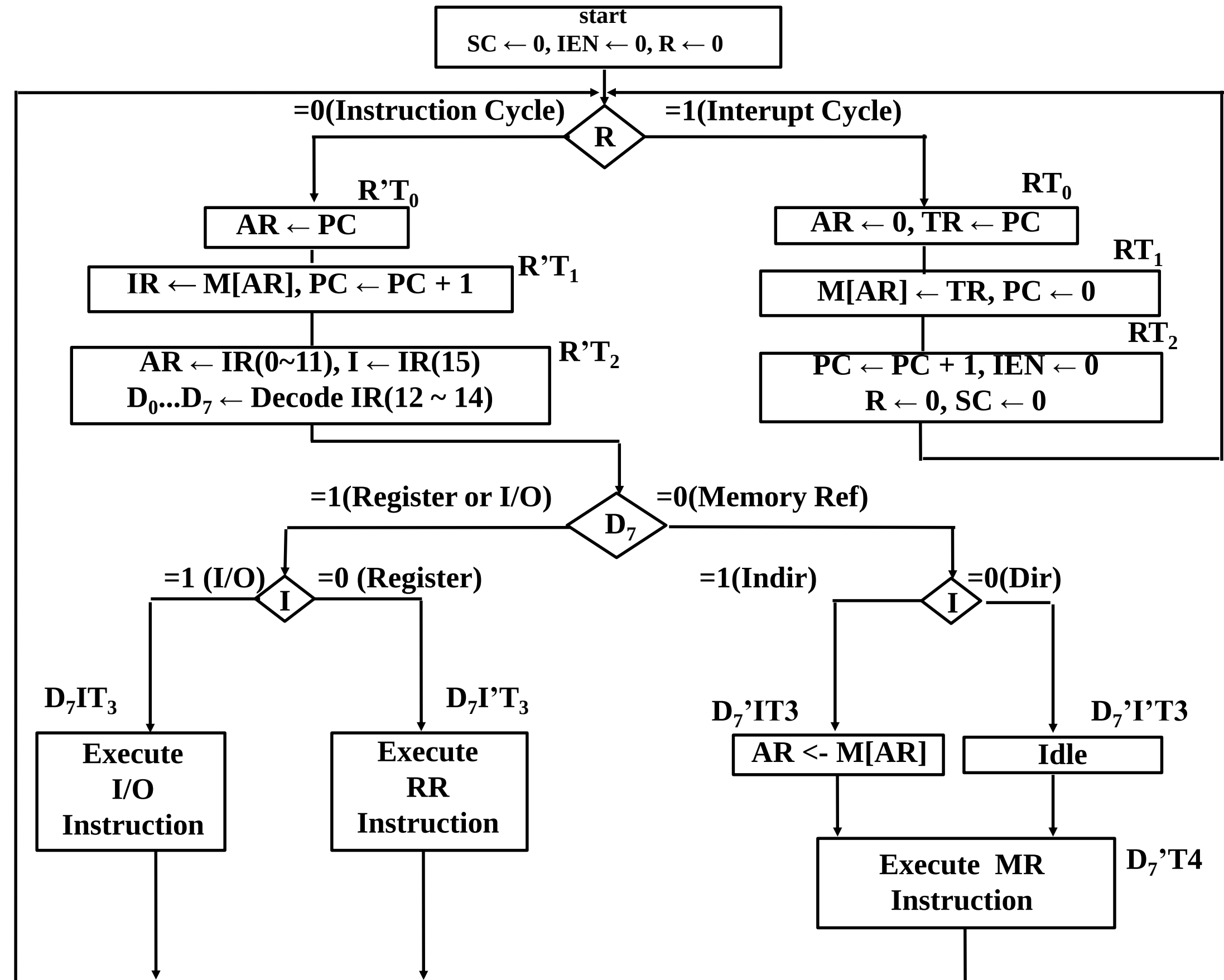
$RT_0: AR \leftarrow 0, TR \leftarrow PC$

$RT_1: M[AR] \leftarrow TR, PC \leftarrow 0$

$RT_2: PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$



Flowchart of Operations



Microoperations

Fetch	$R'T_0:$	$AR \leftarrow PC$
	$R'T_1:$	$IR \leftarrow M[AR], PC \leftarrow PC + 1$
Decode	$R'T_2:$	$D_0, \dots, D_7 \leftarrow \text{Decode } IR(12 \sim 14),$ $AR \leftarrow IR(0 \sim 11), I \leftarrow IR(15)$
Indirect	$D_7'IT_3:$	$AR \leftarrow M[AR]$
Interrupt	$T_0'T_1'T_2'(IEN)(FGI + FGO):$	$R \leftarrow 1$
	$RT_0:$	$AR \leftarrow 0, TR \leftarrow PC$
	$RT_1:$	$M[AR] \leftarrow TR, PC \leftarrow 0$
	$RT_2:$	$PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$
Memory-Reference		
AND	$D_0T_4:$	$DR \leftarrow M[AR]$
	$D_0T_5:$	$AC \leftarrow AC \wedge DR, SC \leftarrow 0$
ADD	$D_1T_4:$	$DR \leftarrow M[AR]$
	$D_1T_5:$	$AC \leftarrow AC + DR, E \leftarrow C_{out}, SC \leftarrow 0$
LDA	$D_2T_4:$	$DR \leftarrow M[AR]$
	$D_2T_5:$	$AC \leftarrow DR, SC \leftarrow 0$
STA	$D_3T_4:$	$M[AR] \leftarrow AC, SC \leftarrow 0$
BUN	$D_4T_4:$	$PC \leftarrow AR, SC \leftarrow 0$
BSA	$D_5T_4:$	$M[AR] \leftarrow PC, AR \leftarrow AR + 1$
	$D_5T_5:$	$PC \leftarrow AR, SC \leftarrow 0$
ISZ	$D_6T_4:$	$DR \leftarrow M[AR]$
	$D_6T_5:$	$DR \leftarrow DR + 1$
	$D_6T_6:$	$M[AR] \leftarrow DR, \text{ if } (DR=0) \text{ then } (PC \leftarrow PC + 1),$ $SC \leftarrow 0$

Microoperations (Cont'd)

Register-Reference

	$D_7I'T_3 = r$ $IR(i) = B_i$ $r:$	(Common to all register-reference instr) ($i = 0,1,2, \dots, 11$) $SC \leftarrow 0$
CLA	$rB_{11}:$	$AC \leftarrow 0$
CLE	$rB_{10}:$	$E \leftarrow 0$
CMA	$rB_9:$	$AC \leftarrow AC'$
CME	$rB_8:$	$E \leftarrow E'$
CIR	$rB_7:$	$AC \leftarrow shr\ AC, AC(15) \leftarrow E, E \leftarrow AC(0)$
CIL	$rB_6:$	$AC \leftarrow shl\ AC, AC(0) \leftarrow E, E \leftarrow AC(15)$
INC	$rB_5:$	$AC \leftarrow AC + 1$
SPA	$rB_4:$	If($AC(15) = 0$) then ($PC \leftarrow PC + 1$)
SNA	$rB_3:$	If($AC(15) = 1$) then ($PC \leftarrow PC + 1$)
SZA	$rB_2:$	If($AC = 0$) then ($PC \leftarrow PC + 1$)
SZE	$rB_1:$	If($E = 0$) then ($PC \leftarrow PC + 1$)
HLT	$rB_0:$	$S \leftarrow 0$

Input-Output

	$D_7IT_3 = p$ $IR(i) = B_i$ $p:$	(Common to all input-output instructions) ($i = 6,7,8,9,10,11$) $SC \leftarrow 0$
INP	$pB_{11}:$	$AC(0-7) \leftarrow INPR, FGI \leftarrow 0$
OUT	$pB_{10}:$	$OUTR \leftarrow AC(0-7), FGO \leftarrow 0$
SKI	$pB_9:$	If($FGI = 1$) then ($PC \leftarrow PC + 1$)
SKO	$pB_8:$	If($FGO = 1$) then ($PC \leftarrow PC + 1$)
ION	$pB_7:$	$IEN \leftarrow 1$
IOF	$pB_6:$	$IEN \leftarrow 0$

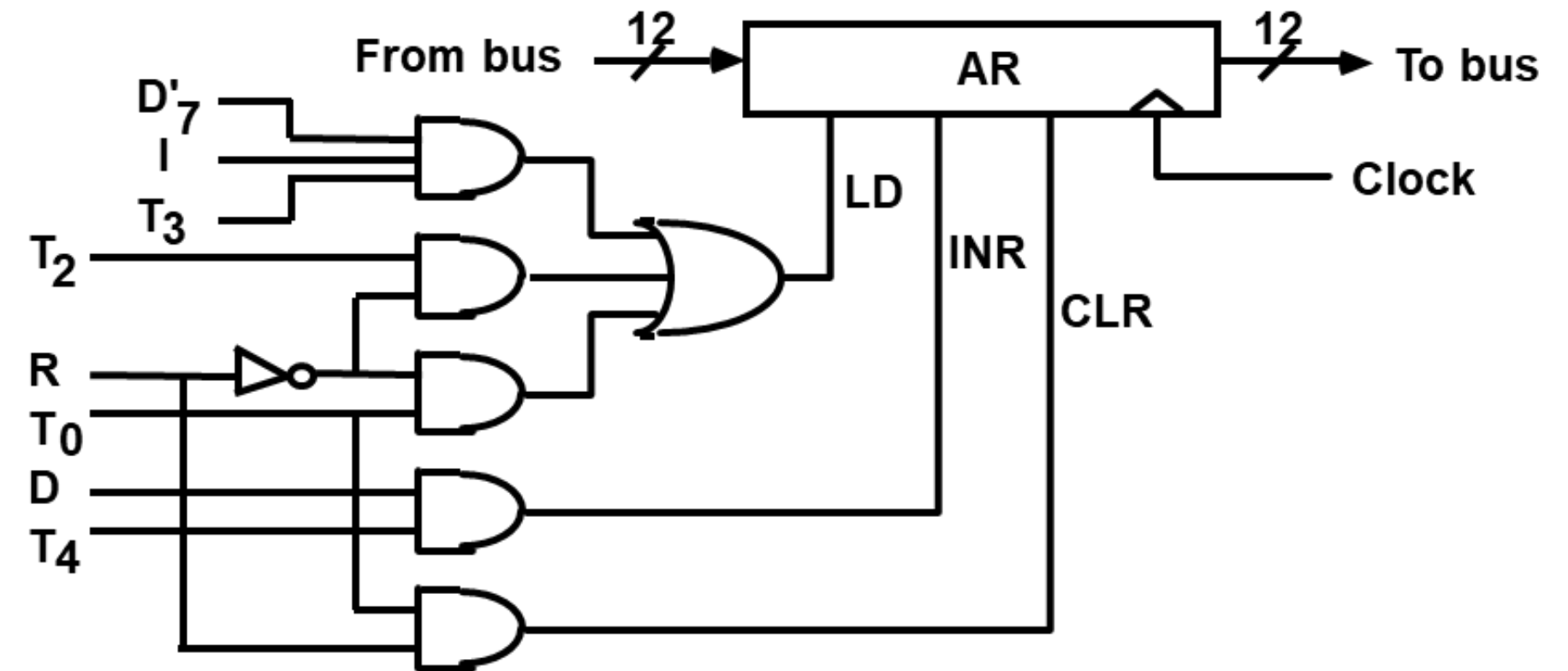
- Hardware components of a basic computer:
 - A memory unit: 4096 x 16
 - Registers → AR, PC, DR, AC, IR, TR, OUTF, INPR, and SC
 - Flip-flops (status) → I, S, E, R, IEN, FGI, and FGO
 - Decoders → 3x8 opcode decoder, 4x16 timing decoder
 - Common bus → 16 bits
 - Control logic gates → Adder and Logic circuit
- Control logic gates:
 - Input Controls of the nine registers
 - Read and Write Controls of memory
 - Set, Clear, or Complement Controls of the flip-flops
 - S_0 , S_1 , S_2 Controls to select a register for the bus
 - AC, and Adder and Logic circuit

- Address Register: AR
 - Scan all of the register transfer statements that change the content of AR

$R'T_0$:	$AR \leftarrow PC$	$LD(AR)$
$R'T_2$:	$AR \leftarrow IR(0-11)$	$LD(AR)$
D'_7IT_3 :	$AR \leftarrow M[AR]$	$LD(AR)$
RT_0 :	$AR \leftarrow 0$	$CLR(AR)$
D_5T_4 :	$AR \leftarrow AR + 1$	$INR(AR)$



$LD(AR) = R'T_0 + R'T_2 + D'_7IT_3$
$CLR(AR) = RT_0$
$INR(AR) = D_5T_4$



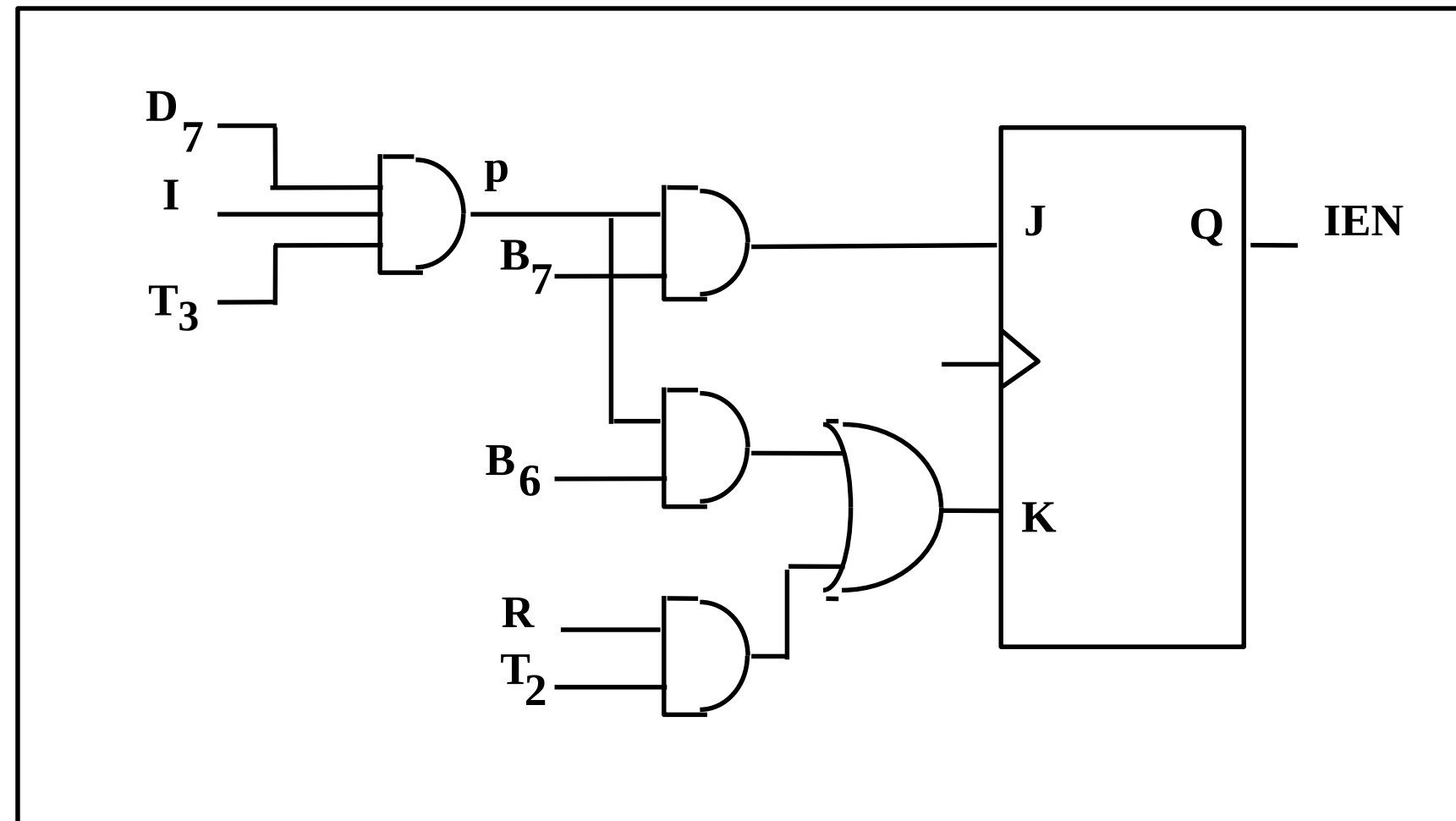
- IEN: Interrupt Enable Flag

$p = D_7IT_3$ (Input/Output Instruction)

pB_7 : $IEN \leftarrow 1$ (I/O Instruction)

pB_6 : $IEN \leftarrow 0$ (I/O Instruction)

RT_2 : $IEN \leftarrow 0$ (Interrupt)



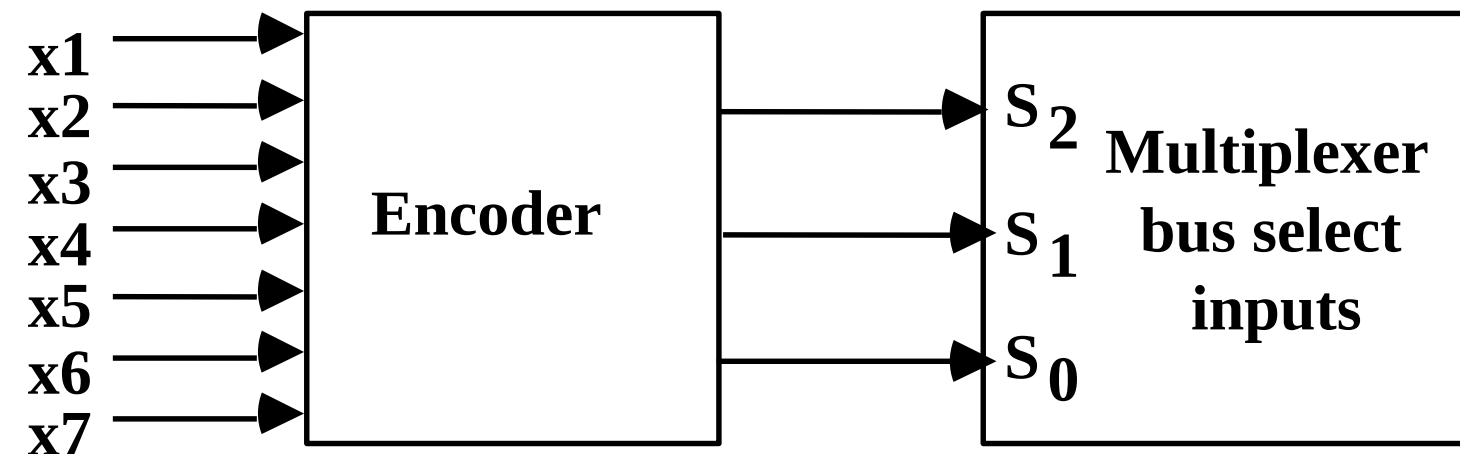
Control of Comon Bus

x1 x2 x3 x4 x5 x6 x7	S2 S1 S0	selected register
0 0 0 0 0 0 0	0 0 0	none
1 0 0 0 0 0 0	0 0 1	AR
0 1 0 0 0 0 0	0 1 0	PC
0 0 1 0 0 0 0	0 1 1	DR
0 0 0 1 0 0 0	1 0 0	AC
0 0 0 0 1 0 0	1 0 1	IR
0 0 0 0 0 1 0	1 1 0	TR
0 0 0 0 0 0 1	1 1 1	Memory

$$S_0 = x_1 + x_3 + x_5 + x_7$$

$$S_1 = x_2 + x_3 + x_6 + x_7$$

$$S_2 = x_4 + x_5 + x_6 + x_7$$



For AR

$D_4T_4: PC \leftarrow AR$
 $D_5T_5: PC \leftarrow AR$

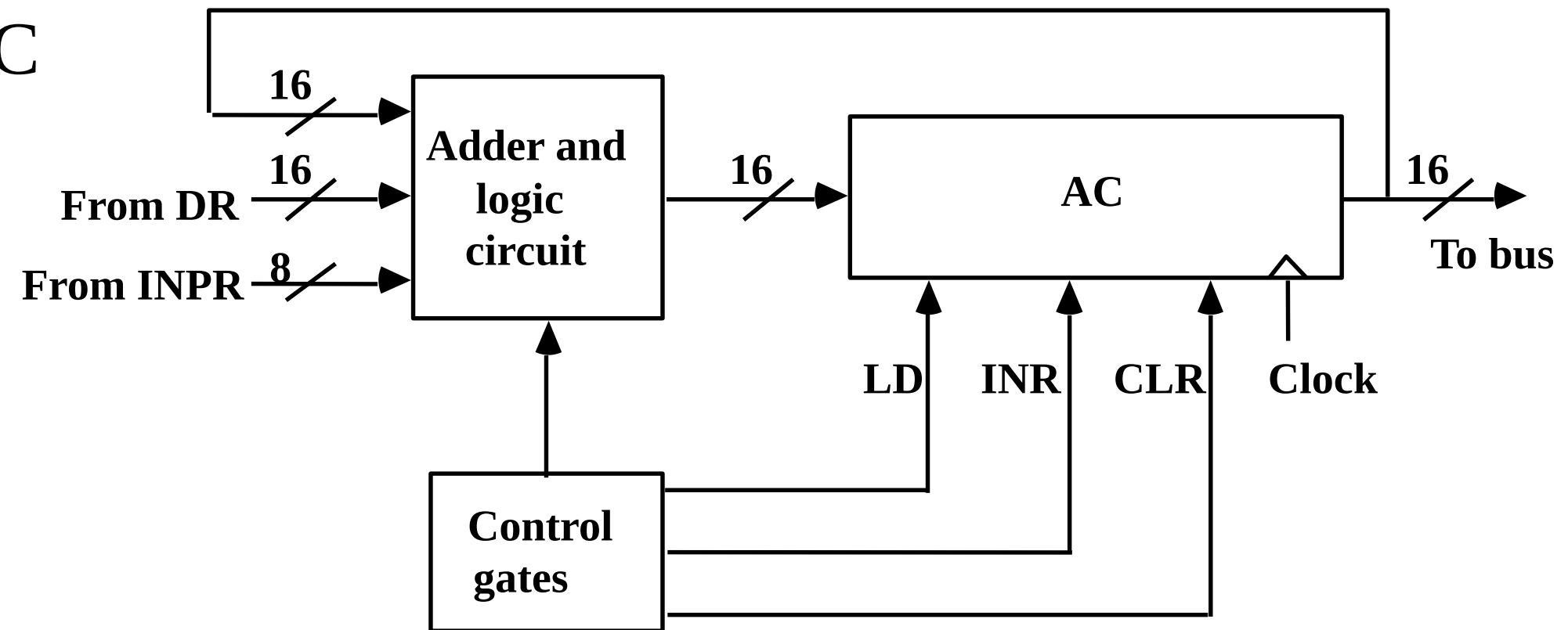


$$x1 = D_4T_4 + D_5T_5$$

$$x_7 = R'T_1 + D_7IT_3 + (D_0 + D_1 + D_2 + D_6)T_4$$

Design of Accumulator Logic

- Circuits associated with AC

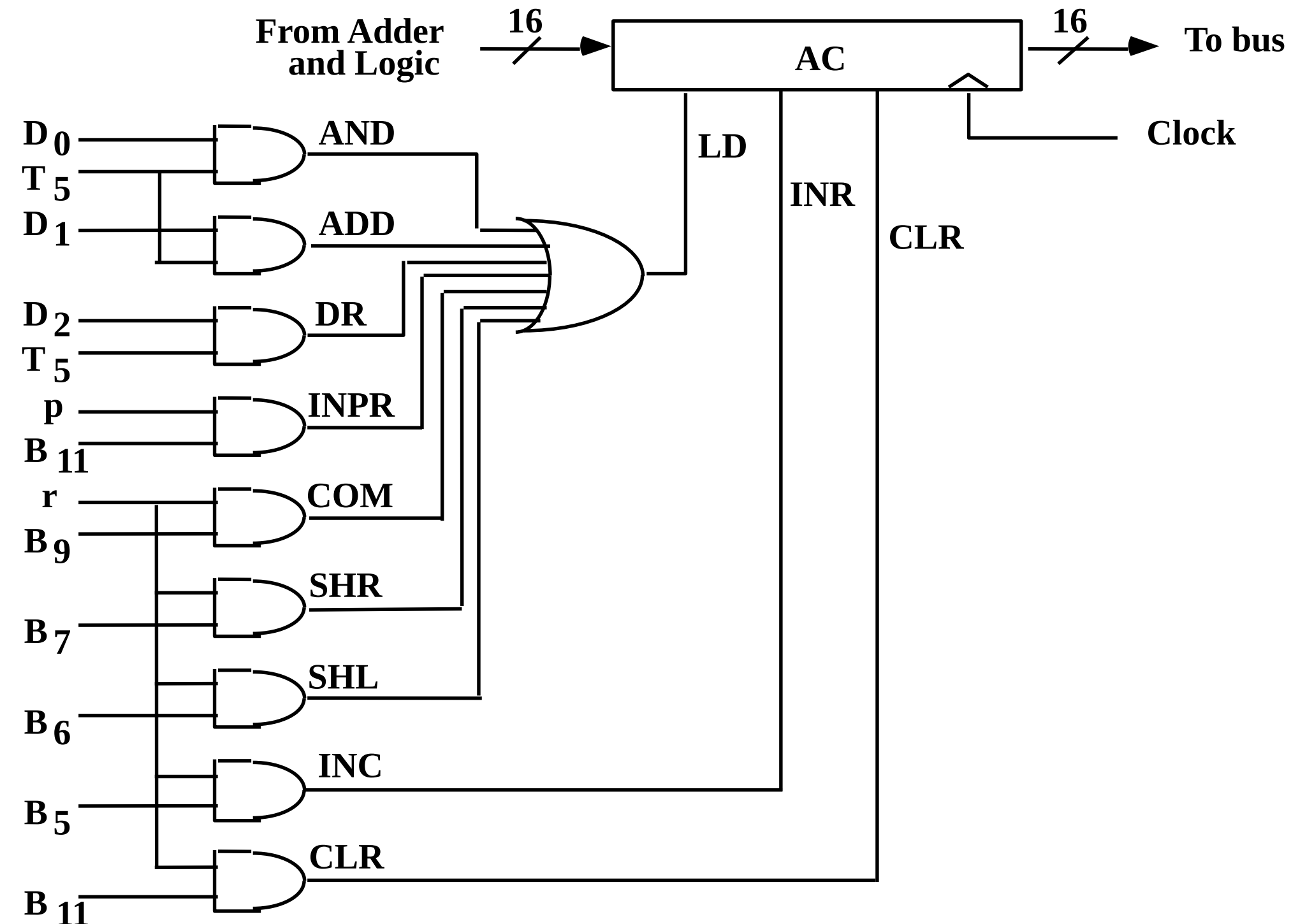


- All the statements that change the content of AC

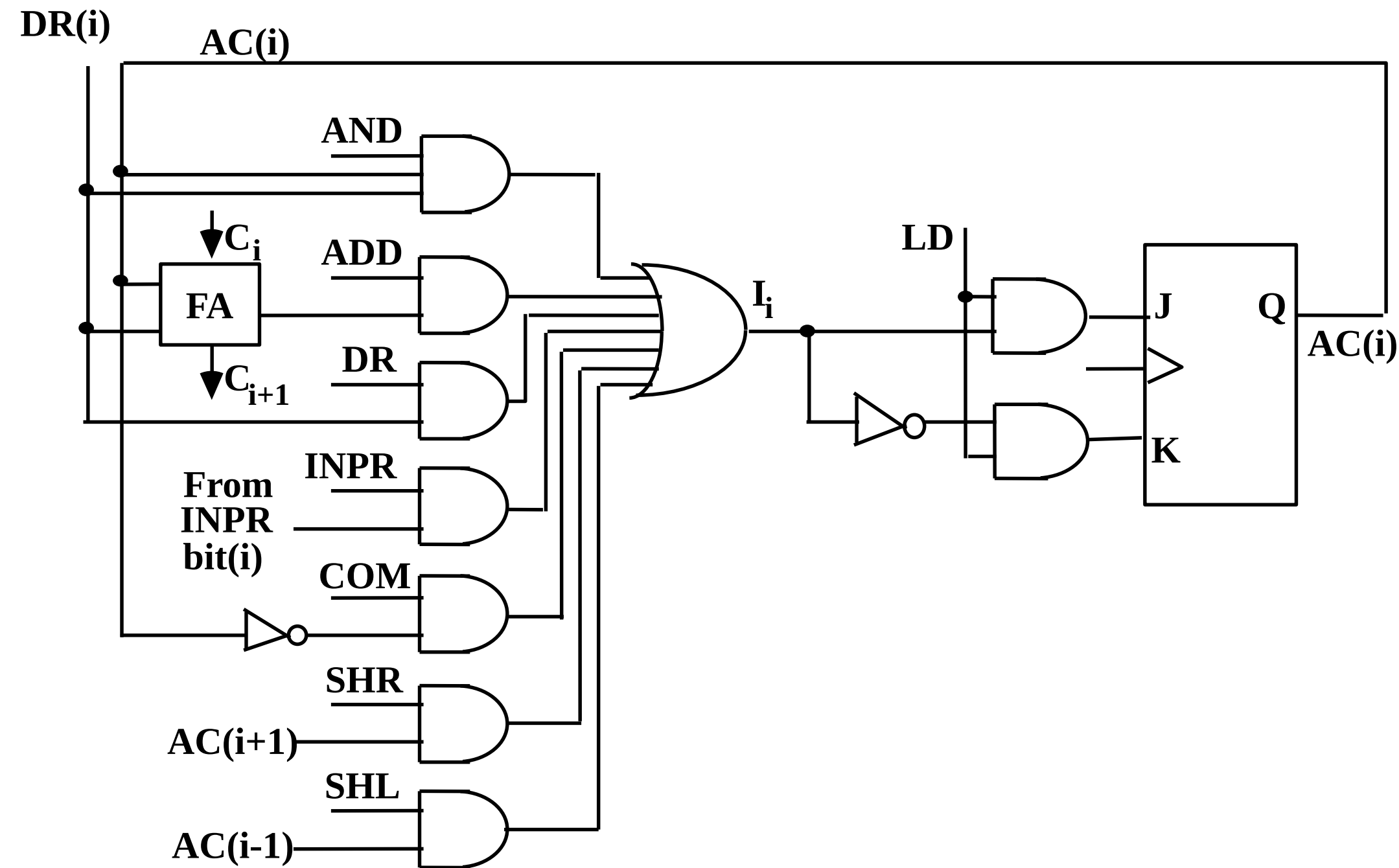
D_0T_5	$AC \leftarrow AC \wedge DR$	AND with DR
D_1T_5	$AC \leftarrow AC + DR$	Add with DR
D_2T_5	$AC \leftarrow DR$	Transfer from DR
pB_{11}	$AC(0-7) \leftarrow INPR$	Transfer from INPR
rB_9	$AC \leftarrow AC'$	Complement
rB_7	$AC \leftarrow shr\ AC, AC(15) \leftarrow E$	Shift right
rB_6	$AC \leftarrow shl\ AC, AC(0) \leftarrow E$	Shift left
rB_{11}	$AC \leftarrow 0$	Clear
rB_5	$AC \leftarrow AC + 1$	Increment

Control of AC Register

- Gate structures for controlling the LD, INR, and CLR of AC



- One stage of Adder and Logic circuit



- a. $IR \leftarrow M[PC]$
- b. $AC \leftarrow AC + TR$
- c. $DR \leftarrow DR + AC$ (AC does not change)

What are the two instructions needed in the basic computer in order to set the E flip-flop to 1?

The operations to be performed with a flip-flop F (not used in the basic computer) are specified by the following register transfer statements:

$xT_3:$	$F \leftarrow 1$	Set F to 1
$yT_1:$	$F \leftarrow 0$	Clear F to 0
$zT_2:$	$F \leftarrow \bar{F}$	Complement F
$wT_5:$	$F \leftarrow G$	Transfer value of G to F

The content of *PC* in the basic computer is 3AF (all numbers are in hexadecimal). The content of *AC* is 7EC3. The content of memory at address 3AF is 932E. The content of memory at address 32E is 09AC. The content of memory at address 9AC is 8B9F.

- What is the instruction that will be fetched and executed next?
- Show the binary operation that will be performed in the *AC* when the instruction is executed.
- Give the contents of registers *PC*, *AR*, *DR*, *AC*, and *IR* in hexadecimal and the values of *E*, *I*, and the sequence counter *SC* in binary at the end of the instruction cycle.

An output program resides in memory starting from address 2300. It is executed after the computer recognizes an interrupt when FGO becomes a 1 (while $IEN = 1$).

- a. What instruction must be placed at address 1?
- b. What must be the last two instructions of the output program?

Show the contents in hexadecimal of registers *PC*, *AR*, *DR*, *IR*, and *SC* of the basic computer when an ISZ indirect instruction is fetched from memory and executed. The initial content of *PC* is 7FF. The content of memory at address 7FF is EA9F. The content of memory at address A9F is 0C35. The content of memory at address C35 is FFFF. Give the answer in a table with five columns, one for each register and a row for each timing signal. Show the contents of the registers after the positive transition of each clock pulse.



Thanks
